

CORE JAVA

Problem Statements — Complete Solutions

Practicals 02 – 13 | Problems 1 – 24

Practical	Topic	Coverage
Practical 02	Class, Object, Methods, Constructors	Problems 1–2
Practical 03	Java Inheritance	Problems 3–4
Practical 04	Static, Final, Super, This Keyword	Problems 5–6
Practical 05	Polymorphism	Problems 7–8
Practical 06	Abstract Class, Interfaces, Lambda	Problems 9–10
Practical 07	Exception Handling	Problems 11–12
Practical 08	Java Threads	Problems 13–14
Practical 09	Packages and Collections	Problems 15–16
Practical 10	File Handling	Problems 17–18
Practical 11	Java Swing	Problems 19–20
Practical 12	JDBC	Problems 21–22
Practical 13	Java Generics	Problems 23–24

Engineering Exam Preparation Guide

PRACTICAL 02 — CLASS, OBJECT, METHODS, CONSTRUCTORS

Problem 1: Bank Account Management System

Concepts Covered

Classes & Objects, Parameterized Constructor, Method Overloading, Instance Methods

Complete Java Solution

```
// BankAccount.java
public class BankAccount {
    // Member variables
    private int accountNumber;
    private String holderName;
    private double balance;

    // Parameterized constructor
    public BankAccount(int accountNumber, String holderName, double balance) {
        this.accountNumber = accountNumber;
        this.holderName = holderName;
        this.balance = balance;
    }

    // Deposit method
    public void deposit(double amt) {
        if (amt > 0) {
            balance += amt;
            System.out.println("Deposited: Rs." + amt +
                " | New Balance: Rs." + balance);
        } else {
            System.out.println("Invalid deposit amount.");
        }
    }

    // Overloaded deposit with remark
    public void deposit(double amt, String remark) {
        if (amt > 0) {
            balance += amt;
            System.out.println("Deposited: Rs." + amt +
                " [" + remark + "] | New Balance: Rs." + balance);
        }
    }

    // Withdraw method
    public void withdraw(double amt) {
        if (amt > 0 && amt <= balance) {
            balance -= amt;
            System.out.println("Withdrawn: Rs." + amt +
                " | New Balance: Rs." + balance);
        } else if (amt > balance) {
            System.out.println("Insufficient balance!");
        } else {
            System.out.println("Invalid withdrawal amount.");
        }
    }
}
```

```

// Display balance
public void displayBalance() {
System.out.println("Account: " + accountNumber +
" | Holder: " + holderName +
" | Balance: Rs." + balance);
}

// Check minimum balance
public String checkMinBalance() {
if (balance < 1000) {
return "WARNING [Acc " + accountNumber + "]: Balance below minimum (Rs.1000)!";
}
return "Balance is healthy for account " + accountNumber;
}

// Main method
public static void main(String[] args) {
BankAccount acc1 = new BankAccount(1001, "Amit", 5000);
BankAccount acc2 = new BankAccount(1002, "Priya", 800);
BankAccount acc3 = new BankAccount(1003, "Rahul", 1500);

System.out.println("=== Initial Balances ===");
acc1.displayBalance();
acc2.displayBalance();
acc3.displayBalance();

System.out.println("\n=== Operations ===");
acc1.deposit(2000); // simple deposit
acc1.deposit(500, "Salary credit"); // overloaded deposit
acc2.withdraw(700);

System.out.println("\n=== Updated Balances ===");
acc1.displayBalance();
acc2.displayBalance();

System.out.println("\n=== Min Balance Check ===");
System.out.println(acc1.checkMinBalance());
System.out.println(acc2.checkMinBalance()); // Priya: 800-700=100 < 1000
System.out.println(acc3.checkMinBalance());
}
}

```

Expected Output

```

=== Initial Balances ===
Account: 1001 | Holder: Amit | Balance: Rs.5000.0
Account: 1002 | Holder: Priya | Balance: Rs.800.0
Account: 1003 | Holder: Rahul | Balance: Rs.1500.0

=== Operations ===
Deposited: Rs.2000.0 | New Balance: Rs.7000.0
Deposited: Rs.500.0 [Salary credit] | New Balance: Rs.7500.0
Withdrawn: Rs.700.0 | New Balance: Rs.100.0

=== Updated Balances ===
Account: 1001 | Holder: Amit | Balance: Rs.7500.0
Account: 1002 | Holder: Priya | Balance: Rs.100.0

```

```
=== Min Balance Check ===  
Balance is healthy for account 1001  
WARNING [Acc 1002]: Balance below minimum (Rs.1000)!  
Balance is healthy for account 1003
```

Key Concepts Explained

- 1. Parameterized Constructor:** Initializes all three fields when object is created.
- 2. Method Overloading (Compile-time Polymorphism):** deposit(double) and deposit(double, String) have the same name but different parameter lists — Java resolves the correct version at compile time.
- 3. this keyword:** Used inside constructor to distinguish instance variables from parameters with the same name.
- 4. Encapsulation:** Fields are private; accessed only via public methods.

Problem 2: Employee Payroll Calculator

Concepts Covered

Default & Parameterized Constructors, this keyword, Method Overloading, Salary computation

Complete Java Solution

```
// Employee.java  
public class Employee {  
    private int empId;  
    private String name;  
    private double basicSalary;  
    private String department;  
  
    // Default constructor  
    public Employee() {  
        this(0, "Unknown", 0.0, "N/A"); // constructor chaining via this()  
    }  
  
    // Parameterized constructor using 'this' keyword  
    public Employee(int empId, String name, double basicSalary, String department) {  
        this.empId = empId;  
        this.name = name;  
        this.basicSalary = basicSalary;  
        this.department = department;  
    }  
  
    public double calculateHRA() { return basicSalary * 0.20; }  
    public double calculateDA() { return basicSalary * 0.11; }  
    public double calculateNetSalary() { return basicSalary + calculateHRA() + calculateDA(); }  
  
    // Basic payslip  
    public void displayPayslip() {  
        System.out.println("----- PAYSIP -----");  
        System.out.println("Emp ID : " + empId);  
        System.out.println("Name : " + name);  
        System.out.println("Dept : " + department);  
        System.out.println("Basic : Rs." + basicSalary);  
        System.out.println("HRA (20%) : Rs." + calculateHRA());  
    }  
}
```

```

System.out.println("DA (11%) : Rs." + calculateDA());
System.out.println("Net Salary: Rs." + calculateNetSalary());
System.out.println("-----");
}

// Overloaded payslip – shows tax deduction if flag is true
public void displayPayslip(boolean showDeductions) {
displayPayslip();
if (showDeductions) {
double tax = calculateNetSalary() * 0.10;
System.out.println("Tax (10%) : Rs." + tax);
System.out.println("Take Home : Rs." + (calculateNetSalary() - tax));
System.out.println("-----");
}
}

public static void main(String[] args) {
Employee e1 = new Employee(201, "Rohit", 40000, "IT");
Employee e2 = new Employee(202, "Sneha", 60000, "HR");
Employee e3 = new Employee(203, "Karan", 30000, "Finance");

// (b) Display all payslips
e1.displayPayslip();
e2.displayPayslip();
e3.displayPayslip();

// (c) Show deductions for employees with net > 50000
System.out.println("\n=== Payslip with Deductions (Net > 50000) ===");
for (Employee e : new Employee[]{e1, e2, e3}) {
if (e.calculateNetSalary() > 50000)
e.displayPayslip(true);
}
}
}

```

Salary Calculation Trace

Employee	Basic (Rs.)	HRA 20% (Rs.)	DA 11% (Rs.)	Net Salary (Rs.)	Tax 10%?
Rohit (E1)	40,000	8,000	4,400	52,400	No (net=52400 > 50000 — YES)
Sneha (E2)	60,000	12,000	6,600	78,600	Yes — Tax=7860
Karan (E3)	30,000	6,000	3,300	39,300	No

Note: Rohit's net salary (Rs.52,400) also exceeds Rs.50,000, so his deductions are also shown.

PRACTICAL 03 — JAVA INHERITANCE

Problem 3: University Staff Hierarchy

Inheritance Types Demonstrated

Single: Employee extends Person | **Multilevel:** HodFaculty → Faculty → Employee → Person | **Hierarchical:** Faculty & AdminStaff both extend Employee

Complete Java Solution

```
// Person.java – Base class
public class Person {
    String name;
    int age;
    String contactNo;

    public Person(String name, int age, String contactNo) {
        this.name = name;
        this.age = age;
        this.contactNo = contactNo;
    }

    public void displayInfo() {
        System.out.println("Name: " + name + " | Age: " + age +
            " | Contact: " + contactNo);
    }
}

// Employee.java – Single Inheritance (extends Person)
public class Employee extends Person {
    String empId;
    String department;

    public Employee(String name, int age, String contactNo,
        String empId, String department) {
        super(name, age, contactNo); // calls Person constructor
        this.empId = empId;
        this.department = department;
    }

    public void displayEmployeeDetails() {
        displayInfo();
        System.out.println("Emp ID: " + empId + " | Dept: " + department);
    }
}

// Faculty.java – Multilevel Inheritance (extends Employee)
public class Faculty extends Employee {
    String designation;
    String[] subjectsTaught;

    public Faculty(String name, int age, String contactNo,
        String empId, String department,
        String designation, String[] subjectsTaught) {
        super(name, age, contactNo, empId, department);
        this.designation = designation;
    }
}
```

```

this.subjectsTaught = subjectsTaught;
}

public void displayFacultyDetails() {
displayEmployeeDetails();
System.out.print("Designation: " + designation + " | Subjects: ");
for (String s : subjectsTaught) System.out.print(s + " ");
System.out.println();
}
}

// HodFaculty.java – Multilevel (extends Faculty)
public class HodFaculty extends Faculty {
double additionalAllowance;

public HodFaculty(String name, int age, String contactNo,
String empId, String department,
String designation, String[] subjectsTaught,
double additionalAllowance) {
super(name, age, contactNo, empId, department, designation, subjectsTaught);
this.additionalAllowance = additionalAllowance;
}

public void displayHodDetails() {
displayFacultyDetails();
System.out.println("HoD Allowance: Rs." + additionalAllowance);
}
}

// AdminStaff.java – Hierarchical (also extends Employee)
public class AdminStaff extends Employee {
String role;

public AdminStaff(String name, int age, String contactNo,
String empId, String department, String role) {
super(name, age, contactNo, empId, department);
this.role = role;
}

public void displayAdminDetails() {
displayEmployeeDetails();
System.out.println("Role: " + role);
}
}

// Main.java
public class Main {
public static void main(String[] args) {
String[] subjects = {"Java", "OS"};

HodFaculty hod = new HodFaculty("Dr. Mehta", 45, "9876543210",
"F001", "IT", "Professor", subjects, 5000);

AdminStaff admin = new AdminStaff("Suresh", 38, "9988776655",
"A001", "Admin", "Office Clerk");

```

```

System.out.println("=== HoD Faculty (Multilevel) ===");
hod.displayHodDetails();

System.out.println("\n=== Admin Staff (Hierarchical) ===");
admin.displayAdminDetails();
}
}

```

Expected Output

```

=== HoD Faculty (Multilevel) ===
Name: Dr. Mehta | Age: 45 | Contact: 9876543210
Emp ID: F001 | Dept: IT
Designation: Professor | Subjects: Java OS
HoD Allowance: Rs.5000.0

=== Admin Staff (Hierarchical) ===
Name: Suresh | Age: 38 | Contact: 9988776655
Emp ID: A001 | Dept: Admin
Role: Office Clerk

```

Key Concepts

super(): Explicitly calls parent class constructor, passing required arguments up the chain.

Multilevel Chain: HodFaculty → Faculty → Employee → Person. Each child class has access to ALL parent methods.

Hierarchical Inheritance: Both Faculty and AdminStaff independently extend Employee, sharing Employee's attributes but having their own additions.

Problem 4: E-Commerce Product Catalogue

Complete Java Solution

```

// Product.java – Base class
public class Product {
    String productId;
    String name;
    double price;

    public Product(String productId, String name, double price) {
        this.productId = productId;
        this.name = name;
        this.price = price;
    }

    public void displayProduct() {
        System.out.println("ID: " + productId + " | Name: " + name +
            " | Price: Rs." + price);
    }
}

// Electronics.java
public class Electronics extends Product {
    String brand;
    int warrantyYears;
}

```



```

public Electronics(String productId, String name, double price,
String brand, int warrantyYears) {
    super(productId, name, price);
    this.brand = brand;
    this.warrantyYears = warrantyYears;
}

@Override
public void displayProduct() {
    super.displayProduct(); // call parent version
    System.out.println("Brand: " + brand +
    " | Warranty: " + warrantyYears + " years [ELECTRONICS]");
}
}

// Clothing.java
public class Clothing extends Product {
    String size, material;

    public Clothing(String productId, String name, double price,
    String size, String material) {
        super(productId, name, price);
        this.size = size;
        this.material = material;
    }

    @Override
    public void displayProduct() {
        super.displayProduct();
        System.out.println("Size: " + size + " | Material: " + material +
        " [CLOTHING]");
    }
}

// Grocery.java
public class Grocery extends Product {
    String expiryDate;
    double weightKg;

    public Grocery(String productId, String name, double price,
    String expiryDate, double weightKg) {
        super(productId, name, price);
        this.expiryDate = expiryDate;
        this.weightKg = weightKg;
    }

    @Override
    public void displayProduct() {
        super.displayProduct();
        System.out.println("Expiry: " + expiryDate +
        " | Weight: " + weightKg + "kg [GROCERY]");
    }
}

// Main.java

```

```

public class Main {
public static void main(String[] args) {
Product[] catalogue = {
new Electronics("P001","Samsung Galaxy", 25000,"Samsung", 2),
new Electronics("P002","Sony Headphones",3500,"Sony",1),
new Clothing("C001","Blue Shirt", 799,"M","Cotton"),
new Clothing("C002","Black Jeans",1299,"L","Denim"),
new Grocery("G001","Basmati Rice", 120,"Dec-2025",1.0),
new Grocery("G002","Sunflower Oil", 180,"Jun-2025",0.5)
};

for (Product p : catalogue) {
System.out.println("-----");
p.displayProduct();
}
}
}

```

PRACTICAL 04 — STATIC, FINAL, SUPER AND THIS KEYWORD

Problem 5: College Admission Counter System

Concepts: static variable, final constant, this(), super(), constructor chaining

```
// Person.java
public class Person {
    String personName;
    int personAge;

    public Person(String name, int age) {
        this.personName = name;
        this.personAge = age;
    }
}

// Student.java
public class Student extends Person {
    String name;
    int rollNo;
    String branch;

    // final constant – cannot be reassigned
    final int MAX_CAPACITY = 60;

    // static counter – shared across all objects
    static int totalAdmitted = 0;

    // Default constructor – uses this() chaining
    public Student() {
        this("Unknown", 0, "General"); // calls parameterized constructor
    }

    // Parameterized constructor
    public Student(String name, int rollNo, String branch) {
        super(name, 18); // calls Person constructor
        this.name = name;
        this.rollNo = rollNo;
        this.branch = branch;
        totalAdmitted++; // auto-increment on every new object
    }

    public static int getTotalAdmitted() { return totalAdmitted; }

    public void displayStudent() {
        System.out.println("Roll: " + rollNo + " | Name: " + name +
            " | Branch: " + branch +
            " | MAX_CAPACITY: " + MAX_CAPACITY);
    }

    public static void main(String[] args) {
        Student s1 = new Student("Asha", 1, "CS");
        Student s2 = new Student("Raj", 2, "IT");
        Student s3 = new Student("Priya", 3, "CS");
        Student s4 = new Student("Dev", 4, "ENTC");
    }
}
```

```

Student s5 = new Student("Neha", 5, "IT");

System.out.println("=== Student Records ===");
s1.displayStudent();
s2.displayStudent();
s3.displayStudent();
s4.displayStudent();
s5.displayStudent();

System.out.println("\nTotal Admitted: " + Student.getTotalAdmitted());

// Attempting to change MAX_CAPACITY would cause compile error:
// s1.MAX_CAPACITY = 80; // ERROR: cannot assign a value to final variable
System.out.println("MAX_CAPACITY is locked at: " + s1.MAX_CAPACITY);
}
}

```

Key Points

static int totalAdmitted: Belongs to class, not instance — incremented each time constructor runs, shared by all objects.

final int MAX_CAPACITY = 60: Cannot be reassigned after initialisation. Attempting `s1.MAX_CAPACITY = 80` gives: *error: cannot assign a value to final variable MAX_CAPACITY*

this(...): Default constructor chains to parameterized constructor, avoiding duplicate initialisation code.

super(name, 18): Explicitly passes values to Person constructor from Student constructor.

Problem 6: Vehicle Registration System

```

// Vehicle.java
public class Vehicle {
    final String vehicleType; // final – cannot be overridden
    int engineCC;

    public Vehicle(String vehicleType, int engineCC) {
        this.vehicleType = vehicleType;
        this.engineCC = engineCC;
    }

    // final method – cannot be overridden in subclass
    public final String getVehicleType() {
        return vehicleType;
    }
}

// RegisteredVehicle.java
public class RegisteredVehicle extends Vehicle {
    String regNo;
    String ownerName;
    static int totalRegistered = 0; // static counter

    public RegisteredVehicle(String vehicleType, int engineCC,
        String regNo, String ownerName) {
        super(vehicleType, engineCC); // super() to init parent
        this.regNo = regNo; // 'this' differentiates param from field
    }
}

```

```

this.ownerName = ownerName;
totalRegistered++;
}

// Attempting to override final method causes compile error:
// public String getVehicleType() { return "..."; }
// ERROR: getVehicleType() in Vehicle cannot be overridden; overriding method is final

public static int getTotalRegistered() { return totalRegistered; }

public void displayRegistration() {
System.out.println("Type: " + getVehicleType() +
" | Engine: " + engineCC + "cc" +
" | Reg No: " + regNo +
" | Owner: " + ownerName);
}

public static void main(String[] args) {
RegisteredVehicle v1 = new RegisteredVehicle("Car", 1500, "MH12AB1234", "Suresh");
RegisteredVehicle v2 = new RegisteredVehicle("Bike", 150, "MH12CD5678", "Pooja");
RegisteredVehicle v3 = new RegisteredVehicle("Truck", 3000, "MH14EF9012", "Ramesh");
RegisteredVehicle v4 = new RegisteredVehicle("Van", 2000, "MH15GH3456", "Sunita");

System.out.println("=== Vehicle Registrations ===");
v1.displayRegistration();
v2.displayRegistration();
v3.displayRegistration();
v4.displayRegistration();
System.out.println("Total Registered: " + RegisteredVehicle.getTotalRegistered());
}
}

```

PRACTICAL 05 — POLYMORPHISM

Problem 7: Banking Transaction Processor

Concepts: Compile-time (Overloading) + Runtime Polymorphism (Overriding), Dynamic Dispatch

```
// Transaction.java – Compile-time Polymorphism (Overloading)
public class Transaction {
    public void processPayment(int amount) {
        System.out.println("Processing int payment: Rs." + amount);
    }
    public void processPayment(double amount) {
        System.out.println("Processing double payment: Rs." + amount);
    }
    public void processPayment(double amount, String remarks) {
        System.out.println("Processing Rs." + amount + " | Remarks: " + remarks);
    }
    public void processPayment(String upiId, double amount) {
        System.out.println("UPI Payment of Rs." + amount + " to " + upiId);
    }
}

// Account.java – Base for Runtime Polymorphism
public class Account {
    double principal;
    public Account(double principal) { this.principal = principal; }

    public double calculateInterest() {
        return 0; // default – overridden in subclasses
    }
}

// SavingsAccount.java
public class SavingsAccount extends Account {
    public SavingsAccount(double p) { super(p); }

    @Override
    public double calculateInterest() {
        return principal * 0.04; // 4% p.a.
    }
}

// FixedDeposit.java
public class FixedDeposit extends Account {
    public FixedDeposit(double p) { super(p); }

    @Override
    public double calculateInterest() {
        return principal * 0.07; // 7% p.a.
    }
}

// Main.java
public class Main {
    public static void main(String[] args) {
```

```
// (a) Compile-time Polymorphism
Transaction t = new Transaction();
t.processPayment(5000);
t.processPayment(2500.50);
t.processPayment(1000.0, "EMI");
t.processPayment("upi@bank", 3000);

// (b,c) Runtime Polymorphism – base class reference
Account a1 = new SavingsAccount(50000);
Account a2 = new FixedDeposit(100000);

System.out.println("\n=== Interest via Dynamic Dispatch ===");
for (Account a : new Account[]{a1, a2}) {
    System.out.println("Runtime Type : " + a.getClass().getSimpleName());
    System.out.println("Interest : Rs." + a.calculateInterest());
}
}
```

Output & Explanation

```
Processing int payment: Rs.5000
Processing double payment: Rs.2500.5
Processing Rs.1000.0 | Remarks: EMI
UPI Payment of Rs.3000.0 to upi@bank

=== Interest via Dynamic Dispatch ===
Runtime Type : SavingsAccount
Interest : Rs.2000.0 (50000 * 0.04)
Runtime Type : FixedDeposit
Interest : Rs.7000.0 (100000 * 0.07)
```

Dynamic Dispatch: Account reference 'a' points to SavingsAccount/FixedDeposit object. JVM calls the overridden version at runtime — not the base class version.

Problem 8: Smart Home Device Control

```
// DeviceController.java – Compile-time Overloading
public class DeviceController {
    public void setTimer(int minutes) {
        System.out.println("Timer set for " + minutes + " minutes.");
    }
    public void setTimer(int hours, int minutes) {
        System.out.println("Timer set for " + hours + "hr " + minutes + "min.");
    }
    public void setTimer(String schedule) {
        System.out.println("Scheduled at: " + schedule);
    }
}

// SmartDevice.java
public class SmartDevice {
    public void activate() {
        System.out.println("Generic device activated.");
    }
}
```

```

// SmartLight.java
public class SmartLight extends SmartDevice {
    int brightness;
    public SmartLight(int b) { this.brightness = b; }

    @Override
    public void activate() {
        System.out.println("Turning lights ON at " + brightness + "% brightness.");
    }
}

// SmartAC.java
public class SmartAC extends SmartDevice {
    int temperature;
    public SmartAC(int t) { this.temperature = t; }

    @Override
    public void activate() {
        System.out.println("Setting AC to " + temperature + " degrees C.");
    }
}

// SmartFan.java
public class SmartFan extends SmartDevice {
    int speed;
    public SmartFan(int s) { this.speed = s; }

    @Override
    public void activate() {
        System.out.println("Fan speed set to " + speed + ".");
    }
}

// Main.java
public class Main {
    public static void main(String[] args) {
        DeviceController dc = new DeviceController();
        dc.setTimer(30);
        dc.setTimer(1, 30);
        dc.setTimer("Morning 6AM");

        // Polymorphic array
        SmartDevice[] devices = {
            new SmartLight(75),
            new SmartAC(22),
            new SmartFan(3)
        };

        System.out.println("\n=== Device Activation (Polymorphism) ===");
        for (SmartDevice d : devices) {
            d.activate(); // runtime dispatch

            // instanceof to identify type
            if (d instanceof SmartLight) System.out.println(" [Type: SmartLight]");
            else if (d instanceof SmartAC) System.out.println(" [Type: SmartAC]");
        }
    }
}

```



```
else if (d instanceof SmartFan) System.out.println(" [Type: SmartFan]");  
}  
}  
}
```

PRACTICAL 06 — ABSTRACT CLASS, INTERFACES, LAMBDA EXPRESSION

Problem 9: Insurance Premium Calculator

Concepts: abstract class, interface, implements, lambda sort, Predicate filter

```
import java.util.*;
import java.util.function.Predicate;
import java.util.stream.Collectors;

// Interface
interface Taxable {
    double applyGST(double premium); // 18% GST
}

// Abstract class
abstract class InsurancePolicy {
    String policyNo;
    String holderName;
    double sumInsured;

    public InsurancePolicy(String pNo, String hName, double sInsured) {
        this.policyNo = pNo;
        this.holderName = hName;
        this.sumInsured = sInsured;
    }

    public abstract double calculatePremium(); // must be overridden

    public void displayPolicy() {
        System.out.println("Policy: " + policyNo + " | Holder: " + holderName +
            " | Sum Insured: Rs." + sumInsured);
        double prem = calculatePremium();
        System.out.println(" Premium: Rs." + prem +
            " | With GST: Rs." + applyGSTWrapper(prem));
    }

    // Helper (calls interface method via concrete class)
    double applyGSTWrapper(double premium) { return premium * 1.18; }
}

// HealthInsurance – 2% premium
class HealthInsurance extends InsurancePolicy implements Taxable {
    public HealthInsurance(String pNo, String hName, double sInsured) {
        super(pNo, hName, sInsured);
    }

    @Override public double calculatePremium() { return sumInsured * 0.02; }

    @Override public double applyGST(double premium) { return premium * 1.18; }
}

// VehicleInsurance – 3% premium
class VehicleInsurance extends InsurancePolicy implements Taxable {
    public VehicleInsurance(String pNo, String hName, double sInsured) {
```

```

super(pNo, hName, sInsured);
}

@Override public double calculatePremium() { return sumInsured * 0.03; }

@Override public double applyGST(double premium) { return premium * 1.18; }
}

// Main
public class InsuranceMain {
public static void main(String[] args) {
List<InsurancePolicy> policies = new ArrayList<>();
policies.add(new HealthInsurance("H001", "Asha", 250000));
policies.add(new HealthInsurance("H002", "Raj", 500000));
policies.add(new HealthInsurance("H003", "Meera", 150000));
policies.add(new VehicleInsurance("V001", "Priya", 150000));
policies.add(new VehicleInsurance("V002", "Dev", 200000));

// Display all
System.out.println("=== All Policies ===");
policies.forEach(InsurancePolicy::displayPolicy);

// Lambda sort by sumInsured ascending
policies.sort((p1, p2) -> Double.compare(p1.sumInsured, p2.sumInsured));
System.out.println("\n=== Sorted by Sum Insured ===");
policies.forEach(p -> System.out.println(p.policyNo + ": Rs." + p.sumInsured));

// Lambda Predicate – filter premium > 5000
Predicate<InsurancePolicy> highPremium = p -> p.calculatePremium() > 5000;
List<InsurancePolicy> filtered = policies.stream()
.filter(highPremium).collect(Collectors.toList());
System.out.println("\n=== Premium > Rs.5000 ===");
filtered.forEach(p -> System.out.println(p.policyNo +
" Premium: Rs." + p.calculatePremium()));
}
}

```

Premium Calculation Table

Policy	Holder	Sum Insured	Type	Rate	Premium	With 18% GST
H001	Asha	2,50,000	Health	2%	5,000	5,900
H002	Raj	5,00,000	Health	2%	10,000	11,800
H003	Meera	1,50,000	Health	2%	3,000	3,540
V001	Priya	1,50,000	Vehicle	3%	4,500	5,310
V002	Dev	2,00,000	Vehicle	3%	6,000	7,080

Policies with premium > Rs.5000: H002 (Rs.10,000) and V002 (Rs.6,000)

Problem 10: Food Delivery Order System

```
import java.util.*;
```

```

interface Discountable {
double applyDiscount(double percent);
}

interface Trackable {
String trackOrder();
}

abstract class FoodItem implements Discountable, Trackable {
String itemName;
double price;

public FoodItem(String itemName, double price) {
this.itemName = itemName;
this.price = price;
}

public abstract void prepareItem();

public void displayItem() {
System.out.println(itemName + " - Rs." + price);
}

@Override
public double applyDiscount(double percent) {
return price - (price * percent / 100);
}

@Override
public String trackOrder() {
return "Estimated delivery: 30-45 minutes";
}
}

class VegItem extends FoodItem {
public VegItem(String name, double price) { super(name, price); }

@Override public void prepareItem() {
System.out.println("Preparing Veg: " + itemName);
}
}

class NonVegItem extends FoodItem {
public NonVegItem(String name, double price) { super(name, price); }

@Override public void prepareItem() {
System.out.println("Preparing Non-Veg: " + itemName);
}
}

public class FoodMain {
public static void main(String[] args) {
List<FoodItem> menu = new ArrayList<>();
menu.add(new VegItem("Paneer Butter Masala", 320));
menu.add(new NonVegItem("Chicken Biryani", 280));
menu.add(new VegItem("Dal Tadka", 180));
}
}

```

```
menu.add(new NonVegItem("Mutton Curry", 380));

System.out.println("=== Preparing Orders ===");
menu.forEach(FoodItem::prepareItem);

System.out.println("\n=== Discounted Prices (10% off) ===");
menu.forEach(item ->
System.out.println(item.itemName + ": Rs." +
item.applyDiscount(10)));

System.out.println("\n=== Delivery ETA ===");
menu.forEach(item -> System.out.println(item.trackOrder()));

// Lambda sort by price
menu.sort(Comparator.comparingDouble(i -> i.price));
System.out.println("\n=== Menu Sorted by Price ===");
menu.forEach(item ->
System.out.println(item.itemName + " - Rs." +
item.applyDiscount(10)));
}
}
```

PRACTICAL 07 — EXCEPTION HANDLING

Problem 11: ATM Withdrawal System

Concepts: Custom Checked/Unchecked Exceptions, try-catch-finally, re-throw

```
// Custom Exceptions

// Checked (extends Exception)
class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String msg) { super(msg); }
}

// Unchecked (extends RuntimeException)
class InvalidAmountException extends RuntimeException {
    public InvalidAmountException(String msg) { super(msg); }
}

// Checked
class DailyLimitExceededException extends Exception {
    public DailyLimitExceededException(String msg) { super(msg); }
}

// ATM class
public class ATM {
    private double balance;
    private double dailyWithdrawn = 0;
    private final double DAILY_LIMIT = 20000;

    public ATM(double initialBalance) { this.balance = initialBalance; }

    public void withdraw(double amount)
        throws InsufficientFundsException, DailyLimitExceededException {
        try {
            // Unchecked – no throws needed
            if (amount <= 0 || amount % 100 != 0)
                throw new InvalidAmountException(
                    "Amount must be > 0 and a multiple of 100. Got: " + amount);

            if (dailyWithdrawn + amount > DAILY_LIMIT)
                throw new DailyLimitExceededException(
                    "Daily limit (Rs." + DAILY_LIMIT + ") exceeded!");

            if (amount > balance)
                throw new InsufficientFundsException(
                    "Insufficient funds. Balance: Rs." + balance +
                    ", Requested: Rs." + amount);

            balance -= amount;
            dailyWithdrawn += amount;
            System.out.println("SUCCESS: Rs." + amount + " withdrawn. Balance: Rs." + balance);

        } catch (InsufficientFundsException e) {
            System.out.println("ERROR: " + e.getMessage());
            throw e; // re-throw after logging
        } finally {
```

```

System.out.println("Transaction log saved.");
}
}

public static void main(String[] args) {
    ATM atm = new ATM(15000);

    // Test 1: Valid withdrawal
    try { atm.withdraw(5000); }
    catch (Exception e) { System.out.println("Caught: " + e.getMessage()); }

    // Test 2: Daily limit breach (5000 already withdrawn)
    try { atm.withdraw(12000); }
    catch (DailyLimitExceededException e) { System.out.println("LIMIT: " + e.getMessage()); }
    catch (InsufficientFundsException e) { System.out.println("FUNDS: " + e.getMessage()); }

    // Test 3: Invalid amount
    try { atm.withdraw(-500); }
    catch (InvalidAmountException e) { System.out.println("INVALID: " + e.getMessage()); }
    catch (Exception e) { System.out.println("Other: " + e.getMessage()); }

    // Test 4: Insufficient funds (balance = 10000, request 12000)
    try { atm.withdraw(12000); }
    catch (InsufficientFundsException e) { System.out.println("INSUFFICIENT: " +
    e.getMessage()); }
    catch (DailyLimitExceededException e){ System.out.println("LIMIT: " + e.getMessage()); }
    }
}

```

Expected Output

```

SUCCESS: Rs.5000.0 withdrawn. Balance: Rs.10000.0
Transaction log saved.
LIMIT: Daily limit (Rs.20000.0) exceeded!
Transaction log saved.
INVALID: Amount must be > 0 and a multiple of 100. Got: -500.0
Transaction log saved.
ERROR: Insufficient funds. Balance: Rs.10000.0, Requested: Rs.12000.0
Transaction log saved.
INSUFFICIENT: Insufficient funds. Balance: Rs.10000.0, Requested: Rs.12000.0

```

Checked vs Unchecked Exceptions

Feature	Checked Exception	Unchecked Exception
Extends	Exception	RuntimeException
Must declare?	Yes — throws clause required	No
Must catch?	Yes — compile error if not	Optional
Example here	InsufficientFundsException DailyLimitExceededException	InvalidAmountException
Use case	Recoverable errors (files, DB, network)	Programming bugs (null, bad input)

Problem 12: Student Online Exam Portal

```
// Custom Exceptions
class ExamTimeExpiredException extends Exception {
public ExamTimeExpiredException(String msg) { super(msg); }
}
class InvalidAnswerFormatException extends RuntimeException {
public InvalidAnswerFormatException(String msg) { super(msg); }
}
class AlreadySubmittedException extends Exception {
public AlreadySubmittedException(String msg) { super(msg); }
}

class Student {
String name;
boolean submitted = false;
public Student(String name) { this.name = name; }
}

public class ExamPortal {
private final long ALLOWED_DURATION = 90; // minutes

public void submitExam(Student s, long timeTaken, String[] answers)
throws ExamTimeExpiredException, AlreadySubmittedException {
try {
if (answers == null || answers.length == 0)
throw new InvalidAnswerFormatException("Answers cannot be null or empty.");

for (String ans : answers)
if (ans == null || ans.trim().isEmpty())
throw new InvalidAnswerFormatException("Null/empty answer found.");

if (timeTaken > ALLOWED_DURATION)
throw new ExamTimeExpiredException(
"Time exceeded! Allowed: " + ALLOWED_DURATION +
" min, Taken: " + timeTaken + " min.");

if (s.submitted)
throw new AlreadySubmittedException(
s.name + " has already submitted the exam!");

s.submitted = true;
System.out.println("Exam submitted successfully for " + s.name +
" in " + timeTaken + " minutes.");

} catch (InvalidAnswerFormatException e) {
System.out.println("FORMAT ERROR: " + e.getMessage());
} catch (AlreadySubmittedException | ExamTimeExpiredException e) {
// multi-catch syntax
System.out.println("SUBMISSION ERROR [" +
e.getClass().getSimpleName() + "]: " + e.getMessage());
throw e; // re-throw checked exceptions
} finally {
System.out.println("Session closed for " + s.name + "\n");
}
}

public static void main(String[] args) {
ExamPortal portal = new ExamPortal();
}
```



```
Student raj = new Student("Raj");
String[] validAnswers = {"A","B","C","D","A"};

// Test 1: Valid submission
try { portal.submitExam(raj, 85, validAnswers); }
catch (Exception e) {}

// Test 2: Time expired
try { portal.submitExam(new Student("Raj"), 95, validAnswers); }
catch (Exception e) {}

// Test 3: Null answer
try { portal.submitExam(new Student("Raj"), 50, null); }
catch (Exception e) {}

// Test 4: Double submit
try { portal.submitExam(raj, 40, validAnswers); }
catch (Exception e) {}
}
}
```

PRACTICAL 08 — JAVA THREADS

Problem 13: Railway Ticket Booking — Seat Reservation

Concepts: extends Thread, synchronized method, Thread.sleep(), race condition

```
// SeatReservation.java – Shared resource
public class SeatReservation {
    private int availableSeats = 10;

    // synchronized prevents race condition
    public synchronized void bookSeats(int count, String agentName) {
        System.out.println(Thread.currentThread().getName() +
            " (" + agentName + ") trying to book " + count + " seats...");
        try { Thread.sleep(100); } // simulate network delay
        catch (InterruptedException e) { e.printStackTrace(); }

        if (availableSeats >= count) {
            availableSeats -= count;
            System.out.println("SUCCESS: " + agentName + " booked " + count +
                " seats. Remaining: " + availableSeats);
        } else {
            System.out.println("FAILED: Seats not available for " + agentName +
                " (Requested: " + count + ", Available: " + availableSeats + ")");
        }
    }
}

// BookingAgent.java
public class BookingAgent extends Thread {
    private SeatReservation reservation;
    private int seatsRequired;
    private String agentName;

    public BookingAgent(SeatReservation r, int seats, String name) {
        this.reservation = r;
        this.seatsRequired = seats;
        this.agentName = name;
    }

    @Override
    public void run() {
        reservation.bookSeats(seatsRequired, agentName);
    }
}

// Main.java
public class Main {
    public static void main(String[] args) throws InterruptedException {
        SeatReservation reservation = new SeatReservation();

        // Available: 10 | Requested total: 12
        BookingAgent a1 = new BookingAgent(reservation, 3, "Agent1");
        BookingAgent a2 = new BookingAgent(reservation, 4, "Agent2");
        BookingAgent a3 = new BookingAgent(reservation, 2, "Agent3");
        BookingAgent a4 = new BookingAgent(reservation, 3, "Agent4");
    }
}
```

```

a1.start(); a2.start(); a3.start(); a4.start();

a1.join(); a2.join(); a3.join(); a4.join();

System.out.println("\nAll booking attempts completed.");
}
}

```

Key Thread Concepts

synchronized method: Only one thread can execute bookSeats() at a time — prevents race conditions where two agents might both read availableSeats=3 simultaneously.

Thread.sleep(100): Simulates network/processing delay. Without synchronization, this gap is where race conditions occur.

join(): Main thread waits for all booking agents to finish before printing the final message.

Thread.currentThread().getName(): Returns "Thread-0", "Thread-1", etc., auto-assigned by JVM.

Problem 14: Stock Market Price Feed Simulator

```

import java.util.Random;

// StockUpdater – implements Runnable
public class StockUpdater implements Runnable {
    private String stockName;
    private volatile double price; // volatile for visibility across threads

    public StockUpdater(String name, double initialPrice) {
        this.stockName = name;
        this.price = initialPrice;
    }

    public double getPrice() { return price; }
    public String getStockName() { return stockName; }

    @Override
    public void run() {
        Random rand = new Random();
        for (int i = 1; i <= 5; i++) {
            double change = price * 0.05 * (rand.nextBoolean() ? 1 : -1);
            price += change;
            System.out.printf("[%s] Update %d: Rs.%.2f (Thread: %s, State: RUNNABLE)%n",
                stockName, i, price, Thread.currentThread().getName());
            try { Thread.sleep(500); }
            catch (InterruptedException e) { Thread.currentThread().interrupt(); }
        }
    }
}

// StockMonitor – extends Thread
public class StockMonitor extends Thread {
    private StockUpdater updater;
    private double threshold;
}

```

```

public StockMonitor(StockUpdater u, double t) {
    this.updater = u;
    this.threshold = t;
}

@Override
public void run() {
    for (int i = 0; i < 5; i++) {
        if (updater.getPrice() > threshold) {
            System.out.println("ALERT: " + updater.getStockName() +
                " crossed Rs." + threshold + "! Current: Rs." + updater.getPrice());
        }
        try { Thread.sleep(500); }
        catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }
}

// Main.java
public class Main {
    public static void main(String[] args) throws InterruptedException {
        StockUpdater reliance = new StockUpdater("RELIANCE", 2500);
        StockUpdater infy = new StockUpdater("INFY", 1800);
        StockUpdater tcs = new StockUpdater("TCS", 4000);

        Thread t1 = new Thread(reliance, "Reliance-Thread");
        Thread t2 = new Thread(infy, "Infy-Thread");
        Thread t3 = new Thread(tcs, "TCS-Thread");

        System.out.println("Thread states before start: " + t1.getState()); // NEW

        StockMonitor monitor = new StockMonitor(reliance, 2600);

        t1.start(); t2.start(); t3.start(); monitor.start();

        System.out.println("Thread states after start: " + t1.getState()); // RUNNABLE/TIMED_WAITING

        // join() – wait for all updaters before final prices
        t1.join(); t2.join(); t3.join();

        System.out.println("\n=== Final Prices ===");
        System.out.printf("RELIANCE: Rs.%.2f\n", reliance.getPrice());
        System.out.printf("INFY : Rs.%.2f\n", infy.getPrice());
        System.out.printf("TCS : Rs.%.2f\n", tcs.getPrice());
    }
}

```

volatile: Ensures price is read from main memory (not CPU cache) by all threads — critical for multi-core systems.

Runnable vs Thread: StockUpdater implements Runnable (preferred — allows extending other classes). StockMonitor extends Thread directly.

PRACTICAL 09 — PACKAGES AND COLLECTIONS

Problem 15: Hospital Patient Record System

Package Structure

hospital.model (Patient.java) | hospital.service (PatientService.java) | hospital.main (Main.java)

```
// === hospital/model/Patient.java ===
package hospital.model;

public class Patient {
    public String patientId, name, disease;
    public int age;

    public Patient(String id, String name, int age, String disease) {
        this.patientId = id; this.name = name;
        this.age = age; this.disease = disease;
    }

    @Override public String toString() {
        return patientId + " | " + name + " | Age:" + age + " | " + disease;
    }
}

// === hospital/service/PatientService.java ===
package hospital.service;
import hospital.model.Patient;
import java.util.*;

public class PatientService {
    ArrayList<Patient> patientList = new ArrayList<>();
    HashMap<String, Patient> patientMap = new HashMap<>();
    LinkedList<Patient> queue = new LinkedList<>();
    TreeSet<String> diseases = new TreeSet<>();

    public void addPatient(Patient p) {
        patientList.add(p);
        patientMap.put(p.patientId, p);
        queue.offer(p);
        diseases.add(p.disease);
    }

    public void displayAll() {
        System.out.println("=== All Patients (ArrayList) ===");
        patientList.forEach(System.out::println);
    }

    public void searchById(String id) {
        Patient p = patientMap.get(id); // O(1) lookup
        System.out.println("Search " + id + ": " + (p != null ? p : "Not found"));
    }

    public void nextInQueue() {
        Patient next = queue.poll(); // FIFO
        System.out.println("Next patient: " + (next != null ? next.name : "Queue empty"));
    }
}
```

```

public void displayDiseases() {
    System.out.println("Diseases (sorted): " + diseases);
}
}

// === hospital/main/Main.java ===
package hospital.main;
import hospital.model.Patient;
import hospital.service.PatientService;

public class Main {
    public static void main(String[] args) {
        PatientService svc = new PatientService();
        svc.addPatient(new Patient("P001", "Asha", 32, "Fever"));
        svc.addPatient(new Patient("P002", "Raj", 45, "Diabetes"));
        svc.addPatient(new Patient("P003", "Priya", 28, "Fever"));
        svc.addPatient(new Patient("P004", "Dev", 60, "BP"));
        svc.addPatient(new Patient("P005", "Neha", 35, "Asthma"));

        svc.displayAll();
        System.out.println();
        svc.searchById("P003");
        svc.nextInQueue(); // Asha (first added)
        svc.displayDiseases(); // sorted alphabetically
    }
}

```

Collections Used and Why

Collection	Use	Advantage
ArrayList<Patient>	Store all patients	Dynamic array, O(1) access by index
HashMap<String, Patient>	patientId → Patient lookup	O(1) average search by key
LinkedList<Patient>	Doctor consultation queue	O(1) add/remove at head/tail (FIFO)
TreeSet<String>	Unique disease names	Automatically sorted, no duplicates

Problem 16: Online Shopping Cart System

```

// === store/model/Product.java ===
package store.model;

public class Product {
    public String productId, name, category;
    public double price;

    public Product(String id, String name, double price, String cat) {
        this.productId = id; this.name = name;
        this.price = price; this.category = cat;
    }
}

// === store/cart/ShoppingCart.java ===

```

```

package store.cart;
import store.model.Product;
import java.util.*;

public class ShoppingCart {
    ArrayList<Product> catalogue = new ArrayList<>();
    HashMap<String,Integer> cart = new HashMap<>();
    Stack<String> undoHistory = new Stack<>();
    TreeMap<Double,String> priceIndex = new TreeMap<>();

    public void addToCatalogue(Product p) {
        catalogue.add(p);
        priceIndex.put(p.price, p.name);
    }

    public void addToCart(String productId, int qty) {
        cart.put(productId, qty);
        undoHistory.push(productId);
        System.out.println("Added to cart: " + productId + " x" + qty);
    }

    public void undoLastAdd() {
        if (!undoHistory.isEmpty()) {
            String last = undoHistory.pop();
            cart.remove(last);
            System.out.println("Undo: Removed " + last + " from cart.");
        }
    }

    public void displayCartTotal() {
        double total = 0;
        for (Map.Entry<String,Integer> entry : cart.entrySet()) {
            Product p = catalogue.stream()
                .filter(pr -> pr.productId.equals(entry.getKey()))
                .findFirst().orElse(null);
            if (p != null) total += p.price * entry.getValue();
        }
        System.out.println("Cart Total: Rs." + total);
    }

    public void displayByPrice() {
        System.out.println("Products sorted by price:");
        priceIndex.forEach((price, name) ->
            System.out.println(" Rs." + price + " - " + name));
    }

    // === store/main/Main.java ===
    package store.main;
    import store.model.Product;
    import store.cart.ShoppingCart;

    public class Main {
        public static void main(String[] args) {
            ShoppingCart cart = new ShoppingCart();
            cart.addToCatalogue(new Product("P1","Phone", 25000,"Electronics"));

```

```
cart.addToCatalogue(new Product("P2","Laptop", 65000,"Electronics"));
cart.addToCatalogue(new Product("P3","Earphones", 2000, "Electronics"));
cart.addToCatalogue(new Product("P4","Mouse", 800, "Accessories"));
cart.addToCatalogue(new Product("P5","Keyboard", 1500, "Accessories"));
cart.addToCatalogue(new Product("P6","Monitor", 12000,"Electronics"));

cart.addToCart("P1", 1);
cart.addToCart("P3", 2);
cart.addToCart("P4", 1);
cart.addToCart("P6", 1);

cart.undoLastAdd(); // undo Monitor
cart.displayCartTotal(); // Phone+2*Earphones+Mouse
cart.displayByPrice();
}
}
```


PRACTICAL 10 — FILE HANDLING

Problem 17: Student Result File Management

```
import java.io.*;
import java.util.*;

public class StudentResultManager {

    static String computeGrade(int marks) {
        if (marks >= 90) return "O";
        if (marks >= 75) return "A";
        if (marks >= 60) return "B";
        if (marks >= 40) return "P";
        return "F";
    }

    // (a) List .txt files in directory
    static void listFiles(String dirPath) {
        File dir = new File(dirPath);
        if (!dir.exists()) { System.out.println("Dir not found."); return; }
        File[] files = dir.listFiles(f -> f.getName().endsWith(".txt"));
        System.out.println("=== .txt Files in " + dirPath + " ===");
        if (files != null) for (File f : files) System.out.println(f.getName());
    }

    // (b) Write results to file
    static void writeResults(String filename) throws IOException {
        String[][] students = {
            {"101", "Asha", "85"}, {"102", "Raj", "72"},
            {"103", "Priya", "91"}, {"104", "Dev", "38"}
        };
        try (BufferedWriter bw = new BufferedWriter(new FileWriter(filename))) {
            bw.write("RollNo,Name,Marks,Grade");
            bw.newLine();
            for (String[] s : students) {
                int marks = Integer.parseInt(s[2]);
                String grade = computeGrade(marks);
                bw.write(s[0] + "," + s[1] + "," + s[2] + "," + grade);
                bw.newLine();
            }
        }
        System.out.println("Results written to " + filename);
    }

    // (c) Read and display as table
    static void readResults(String filename) throws IOException {
        System.out.println("\n=== Results Table ===");
        System.out.printf("%-8s %-12s %-8s %-6s\n", "RollNo", "Name", "Marks", "Grade");
        System.out.println("-".repeat(36));
        try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
            String line;
            br.readLine(); // skip header
            while ((line = br.readLine()) != null) {
                String[] parts = line.split(",");
                System.out.printf("%-8s %-12s %-8s %-6s\n",
```

```

parts[0],parts[1],parts[2],parts[3]);
}
}
}

// (d) Append new record
static void appendResult(String filename, String rec) throws IOException {
try (BufferedWriter bw = new BufferedWriter(new FileWriter(filename, true))) {
bw.write(rec); bw.newLine();
}
System.out.println("Appended: " + rec);
}

// (e) Count passes
static void countPasses(String filename) throws IOException {
int count = 0;
try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
String line; br.readLine();
while ((line = br.readLine()) != null)
if (Integer.parseInt(line.split(",")[2]) >= 40) count++;
}
System.out.println("Total Passed: " + count);
}

public static void main(String[] args) throws IOException {
listFiles("results/");
writeResults("results/results.txt");
readResults("results/results.txt");
appendResult("results/results.txt", "105,Neha,65,B");
readResults("results/results.txt");
countPasses("results/results.txt"); // Expected: 4
}
}

```

Problem 18: Daily Expense Tracker Log

```

import java.io.*;
import java.util.*;

public class ExpenseTracker {

// (a) List all files with sizes
static void listFiles(String dirPath) {
File dir = new File(dirPath);
File[] files = dir.listFiles();
System.out.println("=== Files in " + dirPath + " ===");
if (files != null)
for (File f : files)
System.out.println(f.getName() + " (" + f.length() + " bytes)");
}

// (b) Write expense records
static void writeExpenses(String filename) throws IOException {
String[][] records = {
{"01-May", "Food", "250", "Lunch"},

```

```

{"01-May", "Travel", "80", "Bus"},
{"02-May", "Food", "310", "Dinner"},
{"02-May", "Shopping", "1200", "Shirt"},
{"03-May", "Food", "190", "Breakfast"}
};
try (PrintWriter pw = new PrintWriter(new FileWriter(filename))) {
    pw.println("Date,Category,Amount,Description");
    for (String[] r : records)
        pw.println(r[0]+","+r[1]+","+r[2]+","+r[3]);
    }
System.out.println("Expenses written to " + filename);
}

// (c) Read and display as table
static void displayExpenses(String filename) throws IOException {
    System.out.printf("\n%-10s %-12s %-8s %-15s\n", "Date", "Category", "Amount", "Description");
    System.out.println("-".repeat(48));
    try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
        String line; br.readLine();
        while ((line = br.readLine()) != null) {
            String[] p = line.split(",");
            System.out.printf("%-10s %-12s Rs. %-5s %-15s\n", p[0], p[1], p[2], p[3]);
        }
    }
}

// (d) Append new expense
static void appendExpense(String filename, String rec) throws IOException {
    try (FileWriter fw = new FileWriter(filename, true);
        BufferedWriter bw = new BufferedWriter(fw)) {
        bw.write(rec); bw.newLine();
    }
    System.out.println("Appended: " + rec);
}

// (e) Compute stats
static void computeStats(String filename) throws IOException {
    double total = 0, max = 0;
    int foodCount = 0;
    try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
        String line; br.readLine();
        while ((line = br.readLine()) != null) {
            String[] p = line.split(",");
            double amt = Double.parseDouble(p[2]);
            total += amt;
            if (amt > max) max = amt;
            if (p[1].equalsIgnoreCase("Food")) foodCount++;
        }
    }
    System.out.println("Total Expenditure: Rs." + total);
    System.out.println("Max Single Expense: Rs." + max);
    System.out.println("Food Expense Count: " + foodCount);
}

public static void main(String[] args) throws IOException {
    new File("expenses").mkdirs();
}

```

```
listFiles("expenses/");
writeExpenses("expenses/expenses_log.txt");
displayExpenses("expenses/expenses_log.txt");
appendExpense("expenses/expenses_log.txt", "03-May,Travel,50,Auto");
computeStats("expenses/expenses_log.txt");
// Expected: Total=2080, Max=1200, Food Count=3
}
}
```

PRACTICAL 11 — JAVA SWING

Problem 19: Student Registration Form (GUI)

Note: Swing programs require a JDK environment to compile and run. The code below is complete and functional.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class StudentRegistrationForm extends JFrame {

    // Declare all components
    JTextField txtName, txtEmail, txtContact, txtCity;
    JRadioButton rbMale, rbFemale, rbOther;
    ButtonGroup genderGroup;
    JCheckBox cbJava, cbPython, cbWebDev, cbDataSci;
    JComboBox<String> cmbBranch;
    JList<String> lstHobbies;
    JTextArea txtAddress, txtOutput;
    JButton btnSubmit, btnClear, btnExit;

    public StudentRegistrationForm() {
        setTitle("Student Registration Form");
        setSize(700, 750);
        setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
        setLayout(new BorderLayout(10, 10));

        JPanel formPanel = new JPanel(new GridBagLayout());
        GridBagConstraints gbc = new GridBagConstraints();
        gbc.insets = new Insets(5, 5, 5, 5);
        gbc.anchor = GridBagConstraints.WEST;
        gbc.fill = GridBagConstraints.HORIZONTAL;

        // Name
        gbc.gridx=0; gbc.gridy=0;
        formPanel.add(new JLabel("Name*:"), gbc);
        gbc.gridx=1; txtName = new JTextField(20);
        formPanel.add(txtName, gbc);

        // Email
        gbc.gridx=0; gbc.gridy=1;
        formPanel.add(new JLabel("Email*:"), gbc);
        gbc.gridx=1; txtEmail = new JTextField(20);
        formPanel.add(txtEmail, gbc);

        // Contact
        gbc.gridx=0; gbc.gridy=2;
        formPanel.add(new JLabel("Contact:"), gbc);
        gbc.gridx=1; txtContact = new JTextField(20);
        formPanel.add(txtContact, gbc);

        // Gender
        gbc.gridx=0; gbc.gridy=3;
        formPanel.add(new JLabel("Gender:"), gbc);
        JPanel genderPanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 0, 0));
        rbMale=new JRadioButton("Male"); rbFemale=new JRadioButton("Female",true);
```

```

rbOther=new JRadioButton("Other");
genderGroup = new ButtonGroup();
genderGroup.add(rbMale); genderGroup.add(rbFemale); genderGroup.add(rbOther);
genderPanel.add(rbMale); genderPanel.add(rbFemale); genderPanel.add(rbOther);
gbc.gridx=1; formPanel.add(genderPanel, gbc);

// Branch
gbc.gridx=0; gbc.gridy=4;
formPanel.add(new JLabel("Branch:"), gbc);
cmbBranch = new JComboBox<>(new String[]{"CS", "IT", "ENTC", "Mechanical", "Civil"});
gbc.gridx=1; formPanel.add(cmbBranch, gbc);

// Courses
gbc.gridx=0; gbc.gridy=5;
formPanel.add(new JLabel("Courses:"), gbc);
JPanel coursePanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 0, 0));
cbJava=new JCheckBox("Java"); cbPython=new JCheckBox("Python");
cbWebDev=new JCheckBox("Web Dev"); cbDataSci=new JCheckBox("Data Science");
coursePanel.add(cbJava); coursePanel.add(cbPython);
coursePanel.add(cbWebDev); coursePanel.add(cbDataSci);
gbc.gridx=1; formPanel.add(coursePanel, gbc);

// Hobbies (JList multi-select)
gbc.gridx=0; gbc.gridy=6;
formPanel.add(new JLabel("Hobbies:"), gbc);
lstHobbies = new JList<>(new String[]{"Reading", "Sports", "Music", "Coding", "Travel"});
lstHobbies.setSelectionMode(ListSelectionModel.MULTIPLE_INTERVAL_SELECTION);
lstHobbies.setVisibleRowCount(3);
gbc.gridx=1; formPanel.add(new JScrollPane(lstHobbies), gbc);

// Address
gbc.gridx=0; gbc.gridy=7;
formPanel.add(new JLabel("Address:"), gbc);
txtAddress = new JTextArea(3, 20);
gbc.gridx=1; formPanel.add(new JScrollPane(txtAddress), gbc);

// Buttons
btnSubmit = new JButton("Submit");
btnClear = new JButton("Clear");
btnExit = new JButton("Exit");
JPanel btnPanel = new JPanel();
btnPanel.add(btnSubmit); btnPanel.add(btnClear); btnPanel.add(btnExit);
gbc.gridx=0; gbc.gridy=8; gbc.gridwidth=2;
formPanel.add(btnPanel, gbc);

// Output area
txtOutput = new JTextArea(6, 50);
txtOutput.setEditable(false);
txtOutput.setFont(new Font("Monospaced", Font.PLAIN, 11));

add(new JScrollPane(formPanel), BorderLayout.CENTER);
add(new JScrollPane(txtOutput), BorderLayout.SOUTH);

// Action Listeners
btnSubmit.addActionListener(e -> onSubmit());
btnClear.addActionListener(e -> onClear());

```

```

btnExit.addActionListener(e -> {
int ans = JOptionPane.showConfirmDialog(this,
"Exit application?", "Confirm", JOptionPane.YES_NO_OPTION);
if (ans == JOptionPane.YES_OPTION) System.exit(0);
});

addWindowListener(new WindowAdapter() {
public void windowClosing(WindowEvent e) { btnExit.doClick(); }
});

setLocationRelativeTo(null);
setVisible(true);
}

private void onSubmit() {
if (txtName.getText().trim().isEmpty() || txtEmail.getText().trim().isEmpty()) {
JOptionPane.showMessageDialog(this,
"Name and Email are required!", "Validation Error",
JOptionPane.WARNING_MESSAGE);
return;
}
String gender = rbMale.isSelected() ? "Male" :
rbFemale.isSelected() ? "Female" : "Other";
StringBuilder courses = new StringBuilder();
if (cbJava.isSelected()) courses.append("Java ");
if (cbPython.isSelected()) courses.append("Python ");
if (cbWebDev.isSelected()) courses.append("WebDev ");
if (cbDataSci.isSelected()) courses.append("DataScience");

StringBuilder hobbies = new StringBuilder();
for (String h : lstHobbies.getSelectedValuesList())
hobbies.append(h).append(" ");

String summary = "=== Registration Summary ===
" +
"Name : " + txtName.getText() + "
" +
"Email : " + txtEmail.getText() + "
" +
"Gender : " + gender + "
" +
"Branch : " + cmbBranch.getSelectedItem() + "
" +
"Courses: " + courses + "
" +
"Hobbies: " + hobbies;
txtOutput.setText(summary);
JOptionPane.showMessageDialog(this, summary, "Registration Details",
JOptionPane.INFORMATION_MESSAGE);
}

private void onClear() {
txtName.setText(""); txtEmail.setText("");
txtContact.setText(""); txtAddress.setText("");
rbFemale.setSelected(true);
cbJava.setSelected(false); cbPython.setSelected(false);

```

```

cbWebDev.setSelected(false); cbDataSci.setSelected(false);
lstHobbies.clearSelection();
cmbBranch.setSelectedIndex(0);
txtOutput.setText("");
}

public static void main(String[] args) {
SwingUtilities.invokeLater(StudentRegistrationForm::new);
}
}

```

Problem 20: Library Book Search and Issue GUI

```

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;

public class LibraryGUI extends JFrame {

    DefaultTableModel tableModel;
    JTable bookTable;
    JTextField txtSearch;
    JComboBox<String> cmbCategory;
    JLabel lblCount;

    String[] columns = {"BookID", "Title", "Author", "Category", "Status"};
    Object[][] data = {
        {"B001", "Core Java", "Schildt", "Technology", "Available"},
        {"B002", "DBMS", "Korth", "Technology", "Available"},
        {"B003", "Fiction", "Chetan", "Fiction", "Available"},
        {"B004", "History", "Romila", "History", "Available"},
        {"B005", "Python", "Lutz", "Technology", "Available"},
        {"B006", "Science", "Hawking", "Science", "Available"},
    };

    public LibraryGUI() {
        setTitle("Library Book Management System");
        setSize(750, 500);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout(8, 8));

        // Menu bar
        JMenuBar menuBar = new JMenuBar();
        JMenu fileMenu = new JMenu("File");
        JMenuItem exitItem = new JMenuItem("Exit");
        exitItem.addActionListener(e -> System.exit(0));
        fileMenu.add(exitItem);
        JMenu helpMenu = new JMenu("Help");
        JMenuItem aboutItem = new JMenuItem("About");
        aboutItem.addActionListener(e ->
            JOptionPane.showMessageDialog(this,
                "Library Management System v1.0  

                Built with Java Swing", "About",
                JOptionPane.INFORMATION_MESSAGE));
    }
}

```



```

helpMenu.add(aboutItem);
menuBar.add(fileMenu); menuBar.add(helpMenu);
setJMenuBar(menuBar);

// Top panel
JPanel topPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
topPanel.add(new JLabel("Search:"));
txtSearch = new JTextField(16);
topPanel.add(txtSearch);
topPanel.add(new JLabel("Category:"));
cmbCategory = new JComboBox<>({
    new String[]{"All", "Fiction", "Science", "History", "Technology"});
topPanel.add(cmbCategory);
JButton btnSearch = new JButton("Search");
topPanel.add(btnSearch);
lblCount = new JLabel("Showing 6 books");
topPanel.add(lblCount);
add(topPanel, BorderLayout.NORTH);

// Table
tableModel = new DefaultTableModel(data, columns) {
    public boolean isCellEditable(int r, int c) { return false; }
};
bookTable = new JTable(tableModel);
bookTable.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
add(new JScrollPane(bookTable), BorderLayout.CENTER);

// Bottom buttons
JPanel btnPanel = new JPanel();
JButton btnIssue = new JButton("Issue Book");
JButton btnReturn = new JButton("Return Book");
JButton btnClear = new JButton("Clear Search");
btnPanel.add(btnIssue); btnPanel.add(btnReturn); btnPanel.add(btnClear);
add(btnPanel, BorderLayout.SOUTH);

// Search action
btnSearch.addActionListener(e -> filterTable());
cmbCategory.addActionListener(e -> filterTable());

// Issue action
btnIssue.addActionListener(e -> {
    int row = bookTable.getSelectedRow();
    if (row < 0) { JOptionPane.showMessageDialog(this, "Select a book!"); return; }
    String status = (String) tableModel.getValueAt(row, 4);
    if ("Issued".equals(status)) {
        JOptionPane.showMessageDialog(this, "Book already issued!");
    } else {
        int ans = JOptionPane.showConfirmDialog(this,
            "Issue '" + tableModel.getValueAt(row, 1) + "'",
            "Confirm Issue", JOptionPane.YES_NO_OPTION);
        if (ans == JOptionPane.YES_OPTION) {
            tableModel.setValueAt("Issued", row, 4);
            JOptionPane.showMessageDialog(this, "Book issued successfully!");
        }
    }
});

```

```

// Return action
btnReturn.addActionListener(e -> {
int row = bookTable.getSelectedRow();
if (row < 0) { JOptionPane.showMessageDialog(this,"Select a book!"); return; }
int ans = JOptionPane.showConfirmDialog(this,
"Return '" + tableModel.getValueAt(row,1) + "'",
"Confirm Return", JOptionPane.YES_NO_OPTION);
if (ans == JOptionPane.YES_OPTION) {
tableModel.setValueAt("Available", row, 4);
JOptionPane.showMessageDialog(this,"Book returned successfully!");
}
});

btnClear.addActionListener(e -> {
txtSearch.setText(""); cmbCategory.setSelectedIndex(0);
filterTable();
});

setLocationRelativeTo(null);
setVisible(true);
}

private void filterTable() {
String query = txtSearch.getText().toLowerCase();
String category = (String) cmbCategory.getSelectedItem();
tableModel.setRowCount(0);
int count = 0;
for (Object[] row : data) {
boolean matchQuery = query.isEmpty() ||
row[1].toString().toLowerCase().contains(query) ||
row[2].toString().toLowerCase().contains(query);
boolean matchCat = "All".equals(category) ||
row[3].toString().equals(category);
if (matchQuery && matchCat) {
tableModel.addRow(row); count++;
}
}
lblCount.setText("Showing " + count + " book(s)");
}

public static void main(String[] args) {
SwingUtilities.invokeLater(LibraryGUI::new);
}
}

```

PRACTICAL 12 — JDBC

Prerequisite: MySQL server running. Replace DB_URL, USER, PASSWORD with your credentials.

Problem 21: Employee Records Management using JDBC

```
import java.sql.*;

public class EmployeeJDBC {
    static final String DB_URL = "jdbc:mysql://localhost:3306/company_db";
    static final String USER = "root";
    static final String PASS = "yourpassword";

    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS)) {
            System.out.println("Connected to MySQL.");

            // (a) Create table
            Statement stmt = conn.createStatement();
            stmt.executeUpdate("DROP TABLE IF EXISTS Employee");
            stmt.executeUpdate(
                "CREATE TABLE Employee (" +
                "EmpId INT PRIMARY KEY, Name VARCHAR(50), " +
                "Department VARCHAR(30), Salary DOUBLE, JoiningDate DATE)");
            System.out.println("Table created.");

            // (b) Insert 5 records using PreparedStatement
            String insertSQL = "INSERT INTO Employee VALUES(?,?,?,?,?)";
            PreparedStatement ps = conn.prepareStatement(insertSQL);
            Object[][] emps = {
                {101, "Asha", "IT", 45000.0, "2022-01-10"},
                {102, "Raj", "HR", 38000.0, "2021-06-15"},
                {103, "Priya", "IT", 52000.0, "2020-03-20"},
                {104, "Dev", "Finance", 41000.0, "2023-08-01"},
                {105, "Neha", "IT", 47000.0, "2022-11-05"}
            };
            for (Object[] e : emps) {
                ps.setInt(1, (int)e[0]); ps.setString(2, (String)e[1]);
                ps.setString(3, (String)e[2]); ps.setDouble(4, (double)e[3]);
                ps.setString(5, (String)e[4]);
                ps.executeUpdate();
            }
            System.out.println("5 employees inserted.");

            // (c) SELECT and DISPLAY all
            System.out.println("\n=== All Employees ===");
            ResultSet rs = stmt.executeQuery("SELECT * FROM Employee");
            System.out.printf("%-6s %-10s %-10s %-10s %-12s\n",
                "EmpId", "Name", "Dept", "Salary", "JoinDate");
            System.out.println("-".repeat(52));
            while (rs.next())
                System.out.printf("%-6d %-10s %-10s %-10.0f %-12s\n",
                    rs.getInt(1), rs.getString(2), rs.getString(3),
                    rs.getDouble(4), rs.getString(5));

            // (d) Batch insert
```

```

ps.setInt(1,106); ps.setString(2,"Arjun");
ps.setString(3,"IT"); ps.setDouble(4,43000);
ps.setString(5,"2024-02-10"); ps.addBatch();
ps.setInt(1,107); ps.setString(2,"Sunita");
ps.setString(3,"HR"); ps.setDouble(4,36000);
ps.setString(5,"2024-03-15"); ps.addBatch();
ps.executeBatch();
System.out.println("\nBatch insert: 2 employees added.");

// (e) UPDATE salary – EmpId=2 (Raj) +15%
stmt.executeUpdate(
"UPDATE Employee SET Salary = Salary*1.15 WHERE EmpId=102");
System.out.println("Raj's salary updated (+15%).");

// (f) DELETE EmpId=5 (Neha)
stmt.executeUpdate("DELETE FROM Employee WHERE EmpId=105");
System.out.println("Neha (EmpId 105) deleted.");

// (g) SELECT IT department
System.out.println("\n=== IT Department ===");
rs = stmt.executeQuery(
"SELECT * FROM Employee WHERE Department='IT'");
while (rs.next())
System.out.printf("%-6d %-10s Rs.%.0f%n",
rs.getInt(1),rs.getString(2),rs.getDouble(4));

} catch (SQLException e) {
System.out.println("SQL Error: " + e.getMessage());
}
}
}

```

Problem 22: Library Book Management using JDBC

```

import java.sql.*;

public class LibraryJDBC {
    static final String DB_URL = "jdbc:mysql://localhost:3306/library_db";
    static final String USER = "root", PASS = "yourpassword";

    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection(DB_URL, USER, PASS)) {
            conn.setAutoCommit(false); // for transactions
            Statement stmt = conn.createStatement();

            // (a) Create tables
            stmt.executeUpdate("DROP TABLE IF EXISTS IssuedBooks");
            stmt.executeUpdate("DROP TABLE IF EXISTS Books");
            stmt.executeUpdate(
                "CREATE TABLE Books(BookId INT PRIMARY KEY," +
                "Title VARCHAR(100),Author VARCHAR(50)," +
                "Category VARCHAR(30),Available INT)");
            stmt.executeUpdate(
                "CREATE TABLE IssuedBooks(IssueId INT PRIMARY KEY AUTO_INCREMENT," +
                "BookId INT,StudentName VARCHAR(50)," +

```

```

"IssueDate DATE,ReturnDate DATE," +
"FOREIGN KEY(BookId) REFERENCES Books(BookId))");
conn.commit();

// (b) Insert books
PreparedStatement psBook = conn.prepareStatement(
"INSERT INTO Books VALUES(?,?,?,?)");
Object[][] books = {
{"B001","Core Java","Schildt","Technology",1},
{"B002","DBMS","Korth","Technology",1},
{"B003","Fiction","Chetan","Fiction",1},
{"B004","History","Romila","History",1},
{"B005","Python","Lutz","Technology",1}
};
for (Object[] b : books) {
psBook.setString(1,(String)b[0]); psBook.setString(2,(String)b[1]);
psBook.setString(3,(String)b[2]); psBook.setString(4,(String)b[3]);
psBook.setInt(5,(int)b[4]); psBook.executeUpdate();
}
conn.commit();
System.out.println("5 books inserted.");

// (c) Display all books
ResultSet rs = stmt.executeQuery("SELECT * FROM Books");
System.out.println("\n=== Books ===");
while(rs.next())
System.out.println(rs.getString(1)+" | "+rs.getString(2)+
" | "+rs.getString(4)+" | Available:"+rs.getInt(5));

// (d) Issue book – transaction
try {
PreparedStatement psIssue = conn.prepareStatement(
"INSERT INTO IssuedBooks(BookId,StudentName,IssueDate) VALUES(?,?,CURDATE())");
psIssue.setString(1,"B001"); psIssue.setString(2,"Asha");
psIssue.executeUpdate();
stmt.executeUpdate("UPDATE Books SET Available=0 WHERE BookId='B001'");
conn.commit();
System.out.println("Book B001 issued to Asha. Transaction committed.");
} catch (SQLException e) {
conn.rollback();
System.out.println("Issue failed. Rolled back: "+e.getMessage());
}

// (e) Return book – transaction
try {
stmt.executeUpdate(
"UPDATE IssuedBooks SET ReturnDate=CURDATE() WHERE IssueId=1");
stmt.executeUpdate("UPDATE Books SET Available=1 WHERE BookId='B001'");
conn.commit();
System.out.println("Book returned. Transaction committed.");
} catch (SQLException e) {
conn.rollback();
System.out.println("Return failed. Rolled back.");
}

// (f) Delete book

```

```

stmt.executeUpdate("DELETE FROM Books WHERE BookId='B003'");
conn.commit();
System.out.println("Book B003 deleted.");

// (g) Display by category
System.out.println("\n=== Technology Books ===");
PreparedStatement psCat = conn.prepareStatement(
    "SELECT * FROM Books WHERE Category=?");
psCat.setString(1, "Technology");
rs = psCat.executeQuery();
while(rs.next())
    System.out.println(rs.getString(1)+" | "+rs.getString(2));

} catch (SQLException e) {
    System.out.println("SQL Error: "+e.getMessage());
}
}
}

```

PRACTICAL 13 — JAVA GENERICS

Problem 23: Generic Product Inventory Comparator

Concepts: Generic methods, bounded wildcards, Generic class, type safety

```
import java.util.*;

public class ProductInventory {

    // (a) Generic method – compare two arrays element-by-element
    public static <T> boolean checkIdentical(T[] arr1, T[] arr2) {
        if (arr1.length != arr2.length) return false;
        for (int i = 0; i < arr1.length; i++)
            if (!arr1[i].equals(arr2[i])) return false;
        return true;
    }

    // (b) Generic wildcard method – sum by parity
    public static String sumByParity(List<? extends Number> numbers) {
        double evenSum = 0, oddSum = 0;
        for (Number n : numbers) {
            int val = n.intValue();
            if (val % 2 == 0) evenSum += n.doubleValue();
            else oddSum += n.doubleValue();
        }
        return "Even Sum: " + (int)evenSum + ", Odd Sum: " + (int)oddSum;
    }

    // (c) Generic class
    static class Inventory<T> {
        private List<T> items = new ArrayList<>();

        public void add(T item) { items.add(item); System.out.println("Added: " + item); }
        public boolean remove(T item) { return items.remove(item); }
        public boolean contains(T item) { return items.contains(item); }
        public void display() { System.out.println("Inventory: " + items); }
    }

    public static void main(String[] args) {

        // (a) Test checkIdentical
        Integer[] i1 = {10,20,30};
        Integer[] i2 = {10,20,30};
        Integer[] i3 = {10,25,30};
        System.out.println("[Integer] Identical: " + checkIdentical(i1, i2)); // true
        System.out.println("[Integer] Identical: " + checkIdentical(i1, i3)); // false

        String[] s1 = {"Apple","Mango","Banana"};
        String[] s2 = {"Apple","Mango","Banana"};
        System.out.println("[String] Identical: " + checkIdentical(s1, s2)); // true

        // (b) sumByParity
        List<Integer> nums = Arrays.asList(1,2,3,4,5,6);
        System.out.println(sumByParity(nums)); // Even:12, Odd:9
    }
}
```

```
// (d) Inventory with 3 types
Inventory<String> nameInv = new Inventory<>();
Inventory<Integer> codeInv = new Inventory<>();
Inventory<Double> priceInv = new Inventory<>();

nameInv.add("Laptop"); nameInv.add("Phone"); nameInv.add("Mouse");
codeInv.add(1001); codeInv.add(1002); codeInv.add(1003);
priceInv.add(65000.0); priceInv.add(25000.0); priceInv.add(800.0);

nameInv.display(); codeInv.display(); priceInv.display();
System.out.println("Contains 'Phone': " + nameInv.contains("Phone"));

// nameInv.add(123); // COMPILER ERROR – type safety!
// Above line demonstrates that Inventory<String> cannot hold Integer
}
```

Output

```
[Integer] Identical: true
[Integer] Identical: false
[String] Identical: true
Even Sum: 12, Odd Sum: 9
Added: Laptop Added: Phone Added: Mouse
Added: 1001 Added: 1002 Added: 1003
Added: 65000.0 Added: 25000.0 Added: 800.0
Inventory: [Laptop, Phone, Mouse]
Inventory: [1001, 1002, 1003]
Inventory: [65000.0, 25000.0, 800.0]
Contains 'Phone': true
```

Problem 24: Generic Student Grade Analyser

```
import java.util.*;

public class GradeAnalyser {

    // (a) Compare records – generic method
    public static <T> boolean compareRecords(T[] batch1, T[] batch2) {
        if (batch1.length != batch2.length) return false;
        for (int i = 0; i < batch1.length; i++)
            if (!batch1[i].equals(batch2[i])) return false;
        return true;
    }

    // (b) Analyse parity of marks
    public static String analyseNumbers(List<? extends Number> marks) {
        double evenSum = 0, oddSum = 0;
        for (Number m : marks) {
            int v = m.intValue();
            if (v % 2 == 0) evenSum += m.doubleValue();
            else oddSum += m.doubleValue();
        }
        return "Even-valued marks sum: " + (int)evenSum +
            ", Odd-valued marks sum: " + (int)oddSum;
    }
}
```



```

}

// (c) Generic GradeBook with Comparable bound
static class GradeBook<T extends Comparable<T>> {
    private List<T> grades = new ArrayList<>();

    public void add(T grade) { grades.add(grade); }

    public T getMax() {
        T max = grades.get(0);
        for (T g : grades) if (g.compareTo(max) > 0) max = g;
        return max;
    }

    public T getMin() {
        T min = grades.get(0);
        for (T g : grades) if (g.compareTo(min) < 0) min = g;
        return min;
    }

    public void displayAll() { System.out.println("Grades: " + grades); }
}

// (e) Generic swap method
public static <T> void swap(T[] arr, int i, int j) {
    T temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
}

public static void main(String[] args) {

    // (a) compareRecords
    String[] b1 = {"Asha","Raj","Priya"};
    String[] b2 = {"Asha","Raj","Dev"};
    System.out.println("Batch match: " + compareRecords(b1, b2)); // false

    Integer[] r1 = {101,102,103};
    Integer[] r2 = {101,102,103};
    System.out.println("Roll match : " + compareRecords(r1, r2)); // true

    Double[] m1 = {75.5, 80.0, 90.0};
    Double[] m2 = {75.5, 80.0, 85.0};
    System.out.println("Marks match: " + compareRecords(m1, m2)); // false

    // (b) analyseNumbers
    List<Integer> marks = Arrays.asList(72,85,90,63,48,76);
    System.out.println(analyseNumbers(marks));
    // 72+90+48=210 (even), 85+63+75=223? 85+63=148 odd; 76 even: even=72+90+48+76=286?
    // 72(even)+85(odd)+90(even)+63(odd)+48(even)+76(even) -> even=286, odd=148

    // (d) GradeBook
    GradeBook<Integer> intGB = new GradeBook<>();
    for (int g : new int[]{85,72,91,63}) intGB.add(g);
    intGB.displayAll();
    System.out.println("Max: " + intGB.getMax() + " | Min: " + intGB.getMin());

    GradeBook<Double> dblGB = new GradeBook<>();

```

```

for (double g : new double[]{3.8,3.5,4.0,3.2}) dblGB.add(g);
dblGB.displayAll();
System.out.println("Max GPA: " + dblGB.getMax() + " | Min GPA: " + dblGB.getMin());

// (e) swap
Integer[] arr = {10,20,30,40,50};
System.out.println("Before swap: " + Arrays.toString(arr));
swap(arr, 1, 3);
System.out.println("After swap: " + Arrays.toString(arr));

String[] sarr = {"Alpha","Beta","Gamma"};
swap(sarr, 0, 2);
System.out.println("String swap: " + Arrays.toString(sarr));
}
}

```

Output

```

Batch match: false
Roll match : true
Marks match: false
Even-valued marks sum: 286, Odd-valued marks sum: 148
Grades: [85, 72, 91, 63]
Max: 91 | Min: 63
Grades: [3.8, 3.5, 4.0, 3.2]
Max GPA: 4.0 | Min GPA: 3.2
Before swap: [10, 20, 30, 40, 50]
After swap: [10, 40, 30, 20, 50]
String swap: [Gamma, Beta, Alpha]

```

Generics Summary Table

Syntax	Meaning	Example
<T>	Type parameter — any class	Inventory<String>, Inventory<Integer>
<T extends Comparable<T>>	Bounded type — must implement Comparable	OrderBook<Integer>
<? extends Number>	Wildcard — any subtype of Number	sumByParity(List<Integer>)
T[]	Generic array parameter	checkIdentical(Integer[], Integer[])